Codec

The Aim of the Project

The goal of the *codec* project is to have a common framework to be used underneath when saving or loading a `Resource`, no matter if we want to use a json file, a mongo db, a Lucene document or whatever.

Having a shared system underneath that provides the functionality to actually save and load an `EObject` will make it easier, when it comes to add a new persistence mechanism, for instance.

The idea is that the final user would have only to care about the actual persistence-related stuff (e.g. if we want to work with mongo we would need to implement the layer that talks to our general framework using the specific of mongo, etc.).

What we use

We use `jackson` under the hood, and we tried to expose as much functionalities as possible, in such a way one has control over the majority of the options there. In addition to that we also added some more options, to be able to customize the serialization/deserialization process as much as possible.

The idea is to have some default configuration and then the user should be allowed to overwrite it at the level of the single save/load operation, by passing the corresponding options.

How does it work?

Everything starts with 3 configurators:

- `CodecFactoryConfigurator`: responsible for setting up a `CodecFactory`
- `ObjectMapperConfigurator`: responsible for setting up an `ObjectMapper`
- `CodecModuleConfigurator`: responsible for setting up a `CodecModule`

These configurators are just service providers, and we have a default implementation for all three of them. The implementations are configurable services, meaning that they can be immediately configured in such a way to have the desired basic behavior which should be applied in the general case. Further customization at the level of the single save/load operation can then be set through the loading/saving options, which can then overwrite what has been previously set through the configurations.

Let's have a look now at the details.

`CodecFactoryConfigurator`

Through this service one can set up the properties of the `JsonFactory`, such as:

- `JsonFactory.Feature`
- `StreamWriteFeature`
- `JsonWriteFeature`
- `StreamReadFeature`
- `JsonReadFeature`

The configuration accepts the following properties:

- `enableFeatures`
- `disableFeatures`

whose values should be arrays of `String` with the names of the properties we want to enable a disable. One can
specify the property either with the full name (e. g. `JsonFactory.Feature.CANONICALIZE_FIELD_NAMES`) or just with the simple name (e.g. `CANONIALIZE_FIELD_NAMES`). If the prefix was specified only that specific feature will be enabled/disabled, otherwise all features with the same name will be enabled/disabled (e.g. features common to both serialization and deserialization). The default values of all these properties are the ones reported in the `jackson` documentation.

Another property to be set in the configuration is:

- `type`

This is set to `json` in the default implementation and specifies the type of implementation we are going
to use. It's a useful property to keep track of the type of the implementation you want to use and can be used also to inject corresponding services in other components.

The other two options that can be set through configuration are:

- `genFactory.target`
- `parserFactory.target`

These take as value a filter like `(type = XXXX)`, where you can specify the type of `CodecGeneratorFactory` and `CodecParserFactory` you want to inject. These are **optional** references; if none is found a default `JsonFactory` will be created, otherwise a `CodecFactory` with the injected generator/parser factories will be created.

The service provider is defined in `org.gecko.codec.configurator.CodecFactoryConfigurator`, while our default implementation is in
`org.gecko.codec.jackson.DefaultCodecFactoryConfigurator`.

`ObjectMapperConfigurator`

Through this service is possible to configure properties that belong to the `Object Mapper`:

- `MapperFeature`
- `SerializationFeature`
- `DeserializationFeature`
- `StreamWriteFeature`
- `JsonWriteFeature`
- `StreamReadFeature`
- `JsonReadFeature`

One can enable/disable a feature via configuration, using the properties

- `enableFeatures`

- `disableFeatures`,

which work as the corresponding ones we saw for the `CodecFactoryConfigurator`.

In addition to that, one can set:

- `dateFormat` (default is "yyyy-MM-dd'T' HH: mm: ss")
- `locale` (default is `en-US`)
- `timeZone` (default is "Europe/Berlin")
- `type` (default `json`, same meaning as for the `CodecFactoryConfigurator`)

When using our default implementations, the `CodecFactoryConfigurator` is injected in the `ObjectMapperConfigurator` so one has to specify the right one to use, with the configuration property

- `codecFactoryConfigurator.target` (takes a filter, e. g. `(type = XXXX)`).

The service provider is defined in `org.gecko.codec.configurator.ObjectMapperConfigurator`, while our default implementation is in `org.gecko.codec.jackson.DefaultObjectMapperConfigurator`.

`CodecModuleConfigurator`

The service provider is defined in `org.gecko.codec.configurator.CodecModuleConfigurator`, while our default implementation is in `org.gecko.codec.jackson.module.DefaultCodecModuleConfigurator`.

This service is responsible for setting up the `CodecModule.Builder`. Through the configuration (`org.gecko.codec.configurator.CodecModuleConfig`) one can set:

- `idKey`: to instruct which keyword to use when serializing the id. Default is `_id`;
- `typeKey`: to instruct which keyword to use when serializing the type. Default is `_type`;
- `superTypeKey`: to instruct which keyword to use when serializing the supertype. Default is `_supertype`;
- `refKey`: to instruct which keyword to use when serializing the reference. Default is `$ref`
- `proxyKey` to instruct which keyword to use when serializing proxies. In the default implementation is `_proxy` (**CURRENTLY NOT USED**);
- `timestampKey`: to instruct which keyword to use when serializing timestamps. In the default implementation is `_timestamp` (**CURRENTLY NOT USED**);
- `serializeDefaultValue`: to instruct the module to serialize default attributes values. Default is `FALSE`;
- `serializeEmptyValue`: to instruct the module weather to serialize or not empty values. This refers to empty lists or one dimensional arrays, and to empty `String`. Default value is `FALSE`;
- `serializeNullValue`: to instruct the module weather to serialize or not `null` values. This refers to `null` lists or one dimensional arrays, and to `null` objects. Default value is `FALSE`;
- `useNamesFromExtendedMetaData`: to specify weather or not to use the name set to through the `EXTENDED_META_DATA` annotation, instead of the `EStructuralFeature` name. Default value is `TRUE`;

- `useId`: option used to instruct the module to serialize the id information. Default value is `TRUE`;
- `idOnTop`: option used to instruct the module on weather the id information should be serialized first or not. If set to `FALSE`, the id information is serialized after the type information. Default value is `TRUE`;
- `serializeIdField`: option used to instruct the module to additionally serialize the id field of an `EObject` as it is. This might be superfluous when using as id strategy the one that uses the id field itself, but it might be useful when the id strategy is set to COMBINED. In our default implementation the default value is `FALSE`;
- `idFeatureAsPrimaryKey`: if it is set to `Boolean.TRUE` the value of the ID information will be used as the primary key if it exists. Default value is `TRUE`;
- `writeEnumLiterals`: option to specify weather `Enumerator` values should be serialized by literals or by name. When deserializing the option should be consistent with what was used during serialization (as it should always be the case). If, for whatever reason, it is not, then the deserialization mechanism will fall back and try to deserialize in both ways. Default is `FALSE`, namely enumerators are serialized by name.
- `serializeType`: option used to instruct the module weather to serialize the type information or not. Default is `TRUE`;
- `serializeSuperTypes`: option used to instruct the module weather to serialize the supertype information or not. If this is set to TRUE but `serializeType` is set to FALSE, then this option is ignored. If this is set to TRUE, only the direct parent will be listed as supertype. For the whole inheritance chain, look at `serializeAllSuperTypes` option. Default is `Boolean.FALSE`;
- `serializeAllSuperTypes`: option used to instruct the module weather to serialize the whole chain of inheritance when serializing supertype information. This option is ignored if either `serializeSuperTypes` or `serializeType` is set to `FALSE`. Default value is `FALSE`;
- `serializeSuperTypesAsArray`: option used to specify weather the supertype information should be written in the form of a `String` array. If set to `FALSE`, then a comma separated `String` is used. This option is ignored if either `serializeSuperTypes` or `serializeType` is set to `FALSE`. Default value is `TRUE`.

The `CodecModelInfo`

Another key ingredient is the `org.gecko.codec.info.CodecModelInfo` service. This is responsible for creating the `PackageCodecInfo` whenever a new `EPackage` is registered. The `PackageCodeInfo` is defined in the `org.gecko.codec.info.model`. For every `Eclassifier`, it contains info about codec annotations that might have been used (e.g, to specify the id strategy on to mark a feature as transient). This info will be them used and merged with the options passed to save/load a resource.

There are several codec model annotations currently supported, which are defined in `org.gecko.codec.constants.CodecAnnotations`:

- `CODEC_INHERIT`: annotation at the `EClassifier` level for specifying that codec annotations on the direct parent should be inherited, even if the parent comes from another `EPackage`. By default only if the parent belongs to the same `EPackage` then the codec annotations are inherited.
- `CODEC_TRANSIENT`: annotation at the `EStructuralFeature` level, for specifying that the feature should not be serialized.
- `CODEC_ID_STRATEGY`: annotation at the `EClassifier` level, for specifying the strategy for constructing the id. Accepted values so far are ID-FIELD and COMBINED. The first simply takes the id attribute, while the other performs a concatenation of the fields marked with the `CODEC_ID_FIELD` annotation with the provided order (set through the `CODEC_ID_ORDER`) and separator (set through the `CODEC_ID_SEPARATOR`).
- `CODEC_ID_FIELD`: to annotate a field as an id field. This is ignored if the id strategy is not set to COMBINED.
- `CODEC_ID_ORDER`: to specify the order of the annotated field when constructing the id. This is ignored if the id strategy is not set to COMBINED or the same feature is not marked with the `CODEC_ID_FIELD` annotation.
- `CODEC_ID_SEPARATOR`: annotation at the `EClassifier` level, to specify the separator to be used when constructing the id with the COMBINED strategy. The default separator value is `-`. This option is ignored if the id strategy is different from COMBINED.
- `CODEC_TYPE_INCLUDE`: annotation at the `EClassifier` level, to specify weather the type information should be serialized or not.
- `CODEC_TYPE_USE`: annotation at the `EClassifier` level, to specify a strategy for serializing the type information. Currently supported values are:
  - `CLASS`: the class name will be used (e.g. `org.gecko.codec.demo.model.person.Person`)
  - `NAME`: the class name will be used (e.g. `Person`)
  - `URI`: the URI will be used (e.g. `http://example.de/person/1.0#//Person`)
- `CODEC_ID_VALUE_WRITER_NAME`: annotation at the `EClassifier` level, to specify a `CodecValueWriter` name to be used when serializing the id field. The actual `CodecValueWriter` object should then be one of the automatically registered ones (see the paragraph on `CodecValueWriter/Reader`) or should be passed through the options when saving a Resource.
- `CODEC_ID_VALUE_READER_NAME`: same as the `CODEC_ID_VALUE_WRITER`, but for deserializing.
- `CODEC_TYPE_VALUE_WRITER_NAME`: same as the `CODEC_ID_VALUE_WRITER`, but for serializing the type information.
- `CODEC_TYPE_VALUE_READER_NAME`: same as the `CODEC_ID_VALUE_READER`, but for deserializing the type information.
- `CODEC_VALUE_WRITER_NAME` annotation at the `EStructuralFeature` level, to specify the name for the `CodecValueWriter` that should be used when serializing that feature. The actual `CodecValueWriter` object should then be one of the automatically registered ones (see the paragraph on `CodecValueWriter/Reader`) or should be passed through the options when saving a Resource.
- `CODEC_VALUE_READER_NAME`: same as `CODEC_VALUE_WRITER_NAME` but for deserialization.

`CodecValueWriter` and `CodecValueReader`

The automatically registered `CodecValueWriter` and `CodecValueReader` are defined in `org.gecko.codec.info.helper.CodecIOHelper`. They are registered based on the type of the `CodecModelInfo`, so not all of them are automatically available for all the `CodecModelInfo`. In particular:

- For the ID field: `CodecIOHelper.DEFAULT_ID_VALUE_READER`, `CodecIOHelper.IDFIELD_VALUE_WRITER`, `CodecIOHelper.DEFAULT_ID_VALUE_WRITER`;
- For the TYPE field: `CodecIOHelper.DEFAULT_ECLASS_READER`, `CodecIOHelper.READ_BY_NAME`, `CodecIOHelper.READ_BY_CLASS`, `CodecIOHelper.URI_WRITER`, `CodecIOHelper.WRITE_BY_NAME`, `CodecIOHelper.WRITE_BY_CLASS_NAME`;
- For the SUPERTYPE: `CodecIOHelper.ALL_SUPERTYPE_WRITER`, `CodecIOHelper.SINGLE_SUPERTYPE_WRITER`.

P.A. For the supertype there is currently no possibility to set a different `CodecValueWriter` or `CodecValueReader`.

The `CodecResource`

The `CodecResource` is an extension of the `org.eclipse.emf.ecore.resource.impl.ResourceImpl`, which allows to combine what has been previously configured (the `CodecFactory`, the `ObjectMapper`, the `CodecModule` and the `CodecModelnfo`), with what the final user actually needs when serializing and deserializing a resource.
Through the options one can use when saving/loading, it is possible to overwrite most of the properties previously configured, allowing to have a general setup for e. g. a `json` serialization and, at the same time, the possibility
to change some default behavior. After, and **only after that**, the `ObjectMapper` and the `CodecModule` are actually created from the respective builders and bound together.

The options one can set when saving/loading a Resource so to overwrite the default behavior can be divided in three main categories:

- `ObjectMapperOptions`: to overwrite options set via the `ObjectMapperConfigurator`;
- `CodecModuleOptions`: to overwrite options set via the `CodecModuleConfigurator`;
- `CodecModelInfoOptions`: to overwrite options set via annotation in the EMF model by the `CodecModelInfo` service.

`ObjectMapperOptions`

These are defined in `org.gecko.codec.constants.ObjectMapperOptions` and are:

- `OBJ_MAPPER_DATE_FORMAT`: to overwrite the `dateFormat` property of the `ObjectMapperConfigurator`;
- `OBJ_MAPPER_LOCALE`: to overwrite the `locale` property of the `ObjectMapperConfigurator`;
- `OBJ_MAPPER_TIME_ZONE`: to overwrite the `timeZone` property of the `ObjectMapperConfigurator`;
- `OBJ_MAPPER_SERIALIZATION_FEATURES_WITH`: to specify a `List` of `com.fasterxml.jackson.databind.SerializationFeature` that should be enabled;

- `OBJ_MAPPER_SERIALIZATION_FEATURES_WITHOUT`: to specify a `List` of `com.fasterxml.jackson.databind.SerializationFeature` that should be disabled;
- `OBJ_MAPPER_DESERIALIZATION_FEATURES_WITH`: to specify a `List` of `com.fasterxml.jackson.databind.DeserializationFeature` that should be enabled;
- `OBJ_MAPPER_DESERIALIZATION_FEATURES_WITHOUT`: to specify a `List` of `com.fasterxml.jackson.databind.DeserializationFeature` that should be disabled;
- `OBJ_MAPPER_FEATURES_WITH`: to specify a `List` of `com.fasterxml.jackson.databind.MapperFeature` that should be enabled;
- `OBJ_MAPPER_FEATURES_WITHOUT`: to specify a `List` of `com.fasterxml.jackson.databind.MapperFeature` that should be disabled;

`CodecModuleOptions`

These are defined in `org.gecko.codec.constants.CodecModuleOptions` and are:

- `CODEC_MODULE_SERIALIZE_DEFAULT_VALUE`: to overwrite the `serializeDefaultValue` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_SERIALIZE_EMPTY_VALUE`: to overwrite the `serializeEmptyValue` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_SERIALIZE_NULL_VALUE`: to overwrite the `serializeNullValue` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_USE_NAMES_FROM_EXTENDED_METADATA`: to overwrite the `useNamesFromExtendedMetadata` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_USE_ID`: to overwrite the `useId` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_ID_ON_TOP`: to overwrite the `idOnTop` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_SERIALIZE_ID_FIELD`: to overwrite the `serializeIdField` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_ID_FEATURE_AS_PRIMARY_KEY`: to overwrite the `idFeatureAsPrimaryKey` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_ID_KEY`: to overwrite the `idKey` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_TYPE_KEY`: to overwrite the `typeKey` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_SUPERTYPE_KEY`: to overwrite the `superTypeKey` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_REFERENCE_KEY`: to overwrite the `refKey` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_PROXY_KEY`: to overwrite the `proxyKey` property of the `CodecModuleConfigurator` (**NOT IMPLEMENTED CURRENTLY**);
- `CODEC_MODULE_TIMESTAMP_KEY`: to overwrite the `timestampKey` property of the `CodecModuleConfigurator` (**NOT IMPLEMENTED CURRENTLY**);
- `CODEC_MODULE_SERIALIZE_TYPE`: to overwrite the `serializeType` property of the `CodecModuleConfigurator`;

- `CODEC_MODULE_SERIALIZE_SUPER_TYPES`: to overwrite the `serializeSuperTypes` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_SERIALIZE_ALL_SUPER_TYPES`: to overwrite the `serializeAllSuperTypes` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_SERIALIZE_SUPER_TYPES_AS_ARRAY`: to overwrite the `serializeSuperTypesAsArray` property of the `CodecModuleConfigurator`;
- `CODEC_MODULE_WRITE_ENUM_LITERAL`: to overwrite the `writeEnumLiteral` property of the `CodecModuleConfigurator`.

`CodecModelInfoOptions`

These are defined in `org.gecko.codec.constants.CodecModelInfoOptions` and are:

- `CODEC_IGNORE_FEATURE_LIST`: to specify a list of `EStructuralFeature` that should be ignored during serialization or deserialization. If an `EStructuralFeature` is marked as `transient` in the model or has been annotated with the `CODEC_TRANSIENT` annotation, it will still be ignored even is it is not present in this list;
- `CODEC_IGNORE_NOT_FEATURE_LIST`: to specify a list of `EStructuralFeature` that should **NOT** be ignored during serialization or deserialization. If an `EStructuralFeature` is marked as `transient` in the model or has been annotated with the `CODEC_TRANSIENT` annotation, it will then be taken into account if present in this list;
- `CODEC_ID_SRATEGY`: to overwrite the `org.gecko.codec.constants.CodecAnnotations.CODEC_ID_STRATEGY` annotation;
- `CODEC_ID_SEPARATOR`: to overwrite the `org.gecko.codec.constants.CodecAnnotations.CODEC_ID_SEPARATOR` annotation;
- `CODEC_ID_FEATURES_LIST`: to specify an ordered list of `EStructuralFeature` to be used when constructing the id, if the id strategy is set to COMBINED. Otherwise it will be ignored.
- `CODEC_ID_VALUE_WRITER_NAME`: to overwrite the `org.gecko.codec.constants.CodecAnnotations.CODEC_ID_VALUE_WRITER_NAME` annotation;
- `CODEC_ID_VALUE_READER_NAME`: to overwrite the `org.gecko.codec.constants.CodecAnnotations.CODEC_ID_VALUE_READER_NAME` annotation;
- `CODEC_TYPE_VALUE_WRITER_NAME`: to overwrite the `org.gecko.codec.constants.CodecAnnotations.CODEC_TYPE_VALUE_WRITER_NAME` annotation;
- `CODEC_TYPE_VALUE_READER_NAME`: to overwrite the `org.gecko.codec.constants.CodecAnnotations.CODEC_TYPE_VALUE_READER_NAME` annotation;
- `CODEC_ID_VALUE_WRITER`: to specify a `CodecValueWriter` object to be used when serializing the id information;
- `CODEC_ID_VALUE_READER`: to specify a `CodecValueReader` object to be used when deserializing the id information;
- `CODEC_TYPE_VALUE_WRITER`: to specify a `CodecValueWriter` object to be used when serializing the type information;
- `CODEC_TYPE_VALUE_READER`: to specify a `CodecValueReader` object to be used when deserializing the type information;

- `CODEC_VALUE_WRITERS_MAP`: a Map, where the keys are of type `EStructuralFeature` and the values are of type `CodecValueWriter`, to specify the `CodecValueWrtier` to use when serializing the corresponding `EStructuralFeautre`;
- `CODEC_VALUE_READERS_MAP`: a Map, where the keys are of type `EStructuralFeature` and the values are of type `CodecValueReader`, to specify the `CodecValueReader` to use when deserializing the corresponding `EStructuralFeautre`;
- `CODEC_TYPE_USE`: to overwrite the `org.gecko.codec.constants.CodecAnnotations.CODEC_TYPE_USE` annotation;
- `CODEC_TYPE_INCLUDE`: to overwrite the `org.gecko.codec.constants.CodecAnnotations.CODEC_TYPE_INCLUDE` annotation. If the `CodecModuleOptions.CODEC_MODULE_SERIALIZE_TYPE` is set to `FALSE` then this option is ignored, even if set to `TRUE`.

As these options can be different for different `EClass`, when saving/loading a Resource, one should actually create a Map, where the keys are the `EClass` and the values are the options for that `EClass`. The Map then should be passed via the saving/loading options with the key `org.gecko.codec.constants.CodecResourceOptions.CODEC_OPTIONS`.

```
Map<String, Object> options = new HashMap<>();
Map<EClass, Map<String, Object>> classOptions = new HashMap<>();
Map<String, Object> personOptions = new HashMap<>();

personOptions.put(CodecModelInfoOptions.CODEC_ID_STRATEGY, "ID_FIELD");
classOptions.put(PersonPackage.eINSTANCE.getPerson(), personOptions);
options.put(CodecResourceOptions.CODEC_OPTIONS, classOptions);
resource.save(options);
```

In addition to all these options, when deserializing, the `org.gecko.codec.constants.CodecResourceOptions.CODEC_ROOT_OBJECT` option should be passed. This accepts as value the `EClass` of the root object that has to be deserialized. This option is **MANDATORY** if there is no type information in the document to be read.

The Serialization/Deserialization Mechanism

The actual serialization/deserialization process starts when the `CodecModule` and the `ObjectMapper` are created, after the saving/loading options have been taken into account and merged to the previously configured options.

At this point the `CodecModule#setupModule` method is called. This inherits from `com.fasterxml.jackson.databind.module.SimpleModule#setupModule(com.fasterxml.jackson.databind.Module.SetupContext)`, and it is where the module registers the serializers and deserializers.

In particular, we are overwriting the `org.eclipse.emfcloud.jackson.databind.ser.EMFSerializers` and `org.eclipse.emfcloud.jackson.databind.deser.EMFDeserializers` behavior, in case an `EObject` is found to be saved/loaded. In such case, indeed, we are redirecting the flow to take our corresponding `CodecEObjectSerializer` and `CodecEObjectDeserializer`.

`CodecEObjectSerializer`

The `CodecEObjectSerializer` is defined in `org.gecko.codec.jackson.databind.ser` and it extends the `com.fasterxml.jackson.databind.JsonSerializer`.

When its `serialize` method is called, the first thing it does is retrieving the corresponding `PackageCodecInfo` from the `CodecModule` and extracting from that the `EClassCodecInfo` which corresponds to the `EObject` it wants to write.

Then the `JsonGenerator` starts writing the object, and, based on the `CodecModule` options, calls the corresponding `CodecInfoSerializer` for the various `CodecInfo` objects (id, type, features, etc).

The actual type of the `JsonGenerator` here depends on the `CodecGeneratorFactory` that has been used when constructing the `JsonFactory`.

The `CodecInfoSerializer` type is defined in `org.gecko.codec.jackson.databind.ser.CodecInfoSerializer` and is just an interface which provide a

```
void serialize(EObject rootObj, JsonGenerator gen, SerializerProvider provider)
throws IOException;
```

method.

There are then several implementations for that:

- `IdCodecIngoSerializer`: responsible for writing the id information;
- `TypeCodecInfoSerializer`: responsible for writing the type information;
- `SuperTypeCodecInfoSerializer`: responsible for writing the supertype information;
- `FeatureCodecInfoSerializer`: responsible for writing `EAttribute`;
- `ReferenceCodecInfoSerializer`: responsible for writing containment and non containment `EReference`;
- `EnumeratorSerializer`: responsible for writing enumerators;
- `OperationCodecInfoSerializer`: responsible for writing `EOperation`

`CodecEObjectDeserializer`

The `CodecEObjectDeserializer` is defined in `org.gecko.codec.jackson.databind.deser` and it extends the `com.fasterxml.jackson.databind.JsonDeserializer`.

When an `EObject` has to be read, the first thing it tries to do is to get the type information from the document, so it can construct the right object. This is done in several ways:

- Look for the `org.gecko.codec.constants.CodecResourceOptions.CODEC_ROOT_OBJECT`: this is mandatory in case the type information is not present in the document to be read. In case of root object then we might get the information directly from here.
- Look for the type of the current attribute from the `DeserializationContext`: this is available in case of contained references;
- Look for the type keyword in the document: if the type information has been serialized this should always bring to a result;

- Look for the root context and retrieve the type of the reference: this might be needed in case of non contained references which have been serialized without type information.

Once the right object has been built, we can go through the other features and call the corresponding `CodecInfoDeserializer`.

The `CodecInfoDeserializer` is defined in `org.gecko.codec.jackson.databind.deser` and is just an interface with two methods:

```
public EObject deserialize(JsonParser jp, DeserializationContext ctxt) throws
IOException;

public void deserializeAndSet(JsonParser jp, EObject current,
DeserializationContext ctxt, Resource resource) throws IOException;
```

The implementations are:

- `IdCodecInfoDeserializer`: responsible for reading the id information and setting the id fields, if the `CodecModule` `serializeIdField` was set to `FALSE`;
- `FeatureCodecInfoDeserializer`: responsible for reading attributes, containment and non containment references, enumerators;
- `ReferenceCodecInfoDeserializer`: it is called from the `FeatureCodecInfoDeserializer` in order to perform the actual deserialization of non containment references.

The Layer in Between

What allows to use this general setup for multiple persistence mechanisms is a layer in between that is formed, on the serialization side, from the `org.gecko.codec.jackson.databind.ser.CodecGeneratorBaseImpl`, and, on the deserialization side, from the `org.gecko.codec.jackson.databind.deser.CodecParserBaseImpl`.

These are `abstract` classes which inherit, respectively, from `com.fasterxml.jackson.core.base.GeneratorBase` and `com.fasterxml.jackson.core.base.ParserBase`. Extending these classes was necessary, from the serialization side, to allow the use of an extension of the `JsonWriteContext` with our `CodecWriteContext`, so that additional information, such as the current `EStructuralFeature`, can be recorded. In addition to that we provide default implementations for methods that write/read, which then further implementations can overwrite.

The idea is that specific codec implementations provide their own implementations of such classes, so that they can put all the logic which belongs to the specific persistence mechanism is there, while the general framework remains the same for every implementations.

Implementations

We currently have implemented the codec for:

- `json`
- `mongodb`

For the `json` implementation we are relying on the `JsonGenerator`s and `JsonParser`s already available in jackson, while for the `mongo` implementation we are providing `org.gecko.condec.mongo.MongoCodecGenerator` and `org.gecko.condec.mongo.MongoCodecParser`, which are then constructed in the corresponding `MongoGeneratorFactory` and `MongoParserFactory`, injected in the `CodecFactoryConfigurator`.

Missing or Not Tested Features

- `proxyKey` and `timestampKey` are available options in the `CodecModuleConfig` but they are not currently used in the implementations of the serialization/deserialization process;
- currently it is only possible to save the supertypes as an array or a comma separated String of URIs; it might be useful to have a strategy like we have for the type information, so one can save the supertypes also as class names or simply as names. It might also be useful to serialize them with another separator rather than a comma separated String (?)
- serialization/deserialization of Maps has not been tested and nothing special has been implemented for them, so not sure if it works out of the box with the "standard" jackson serializers or not.